Task 3: Work with YourTTS – Report

Oct 20, 2026 · @Gedas

> **Note before submitting (delete this box):** text in [square brackets] is a blank for your own results. Run the scripts, copy the printed numbers into the blanks, insert your pictures where it says [Insert …], and change any sentence that does not match what you see or hear. The discussion sentences describe what is typical for YourTTS, so check each one against your own plots.

1. Introduction

The goal of this work was to test how a pretrained YourTTS model handles a language it was not trained on (Lithuanian), and to prepare a small Lithuanian dataset that could later be used to adapt the model.

YourTTS is a multilingual, multi-speaker text-to-speech model. It can do **zero-shot voice cloning**: given a short recording of a new speaker, it produces speech in that speaker's voice without any extra training. The model was loaded with the Coqui TTS library (`tts_models/multilingual/multi-dataset/your_tts`).

The work had four parts:

1. Check which languages the pretrained model supports.
2. Synthesize an English sentence in my own voice and compare it with my natural recording.
3. Do the same for a Lithuanian sentence, using the English language setting.
4. Prepare a one-speaker, one-emotion dataset from the Lithuanian Emotional Speech Corpus (EMO).

Tools: Python 3.10, TTS 0.22.0 (Coqui), PyTorch, librosa (audio loading and F0 estimation), NumPy (FFT), soundfile (audio parameters) and matplotlib (plots). The F0 contours were estimated with the pYIN algorithm (`librosa.pyin`, 60–400 Hz). The magnitude spectrum was computed as the FFT of the whole signal, in dB.

2. Supported languages

The pretrained YourTTS model supports three languages, and **Lithuanian is not one of them**. The command `print(tts.languages)` returned:

```
['en', 'fr-fr', 'pt-br']
```

Code	Language
en	English
fr-fr	French (France)
pt-br	Portuguese (Brazil)

The model was trained on English, French and Brazilian Portuguese data only. So, for Lithuanian text, the model has to be given one of these three language settings. In this work the English setting was used.

3. Zero-shot synthesis: English

I recorded the sentence *"This is a test of the speech synthesis system."* and saved it as `reference_en.wav`. This recording was used both as the natural speech sample and as the voice reference for YourTTS. The model then synthesized the same sentence in my voice (`output_en.wav`).

3.1 Reference recording

Parameter	Value
Duration	[ ] s
Sampling frequency	[ ] Hz
Audio format	WAV, [PCM_16 = 16-bit]
Channels	[1 (mono)]
Recorded with	[phone / laptop microphone / Audacity]

3.2 Quality of the synthesized speech

[Write your own opinion in 3–4 sentences. Questions to answer: Is the sentence easy to understand? Does it sound like you? Does it sound natural or robotic? Are there any glitches, wrong words or strange pauses?]

Example: *The English sentence is fully understandable and every word is pronounced correctly. The voice is similar to mine in pitch, but it sounds smoother and less lively than my recording. The speech sounds slightly muffled, and the rhythm is more even than in natural speech.*

3.3 Plots

[Insert plots_en.png here: waveform, magnitude spectrum and F0 contour; natural speech on the left, synthesized on the right.]

Figure 1. Natural (left) and synthesized (right) English speech: waveform, magnitude spectrum and F0 contour.

3.4 Comparison

Measure	Natural	Synthesized
Duration (s)	[ ]	[ ]
Sampling frequency (Hz)	[ ]	16000
Mean F0 (Hz)	[ ]	[ ]
F0 range (Hz)	[ ]–[ ]	[ ]–[ ]

Duration. The synthesized signal is [shorter / longer] than the natural one ([ ] s vs [ ] s). Part of the difference comes from silence: my recording has [ ] s of silence at the start and end, while YourTTS adds very little. The speaking rate is also different, because the model chooses its own phoneme durations. [Check this against the waveforms.]

Spectrum. YourTTS produces audio at 16 kHz, so its spectrum ends at 8 kHz (half the sampling frequency). My recording at [ ] Hz contains frequencies up to [ ] kHz. As a result, high-frequency sounds such as "s" and "th" carry less energy in the synthesized speech, which can make it sound duller. Below about 4 kHz both spectra have a similar shape, with most energy below 1 kHz. [Check this against the spectrum plots.]

F0. The mean F0 of the synthesized speech ([ ] Hz) is [close to / higher than / lower than] mine ([ ] Hz), so the model [did / did not] copy my voice pitch well. The synthesized F0 contour is [flatter / similar], with a range of [ ] Hz compared with [ ] Hz in my recording. A flatter F0 contour means less intonation, which is typical for TTS and makes speech sound more monotonous. [Check this against the F0 plots.]

4. Zero-shot synthesis: Lithuanian

I recorded the sentence *"Tai yra šnekos sintezės sistemos testas."* and saved it as `reference_lt.wav`. Lithuanian is not supported, so the same sentence was synthesized with `language="en"` (`output_lt.wav`).

4.1 Quality of the synthesized speech

[Write your own opinion in 3–4 sentences. Questions to answer: Can a Lithuanian listener understand the sentence? Which words or sounds are wrong? Does it still sound like your voice? Does it sound like a foreign accent?]

Example: *The Lithuanian sentence is only partly understandable. The model reads the text with English pronunciation rules, so it sounds like an English speaker reading Lithuanian. The letters "š" and "ė" are not in the English vocabulary of the model and are [skipped / pronounced wrongly], so "šnekos" sounds like ["nekos"]. The voice still resembles mine, which shows that voice cloning works even when the language is wrong.*

Tip: when the script runs, the console may show warnings like *Character 'š' not found in the vocabulary. Discarding it.* If you see them, mention them here, because they explain the missing sounds.

4.2 Plots

[Insert plots_lt.png here.]

Figure 2. Natural (left) and synthesized (right) Lithuanian speech: waveform, magnitude spectrum and F0 contour.

4.3 Comparison

Measure	Natural	Synthesized
Duration (s)	[ ]	[ ]
Sampling frequency (Hz)	[ ]	16000
Mean F0 (Hz)	[ ]	[ ]
F0 range (Hz)	[ ]–[ ]	[ ]–[ ]

Duration. The synthesized sentence lasts [ ] s, compared with [ ] s for my recording. [If it is much shorter: this is probably because some letters were dropped, so the model had fewer sounds to say.]

Spectrum. As in the English case, the synthesized spectrum stops at 8 kHz. The Lithuanian sentence has several fricatives ("š", "s", "z"), so the missing high frequencies are [more / less] noticeable here. [Check this against the spectrum plots.]

F0. The mean F0 of the synthesized speech is [ ] Hz, compared with [ ] Hz in my recording. The intonation follows an [English / flat] pattern rather than natural Lithuanian intonation; for example, [describe what you see, e.g. the pitch rises or falls in different places than in

my recording].

English vs Lithuanian. Compared with the English result (Section 3), the Lithuanian result is [clearly worse] in [intelligibility and pronunciation], while the voice similarity is [about the same]. This shows that YourTTS can copy a voice in any language, but it can only pronounce well the languages it was trained on.

5. Lithuanian dataset preparation (EMO corpus)

A training dataset for adapting YourTTS to Lithuanian was prepared from the Lithuanian Emotional Speech Corpus (EMO). The full corpus metadata lists 349,696 transcribed recordings from five speakers (F2, F3, F4, M3, M4) in five emotions: D – joy, L – sadness, N – surprise, P – anger, Z – neutral. Only one archive part, `[emo-wavs.zip.009]`, was downloaded.

5.1 Method

1. The archive part was extracted into the folder `emo_wavs`.
2. Speaker and emotion were read from the file names. For example, `0301-F3Z-0001.wav` means speaker F3 and emotion Z (neutral).
3. All recordings of one speaker in one emotion were selected.
4. The transcription of each recording was taken from the file `metadata_of_EMO_dataset.csv`.
5. The files were copied to `dataset/wavs/` and renamed `0001.wav`, `0002.wav`, …
6. The recordings were shuffled randomly (seed 42) and split into 90 % training and 10 % validation.

The metadata files use the Coqui "ljspeech" format: file name without `.wav` | transcription | normalized transcription. For example:

```
0001|lietuvos kurorte neringoje šią vasarą jau dvidešimt septintą kartą vyks
...|lietuvos kurorte ...
```

The resulting folder structure:

```
dataset/
├── wavs/
│   ├── 0001.wav
│   ├── 0002.wav
│   └── ...
├── metadata_train.csv
└── metadata_val.csv
```

5.2 Selected data

Item	Value
Speaker	[F3] ([female])
Emotion	[Z – neutral (Neutrali)]
Number of recordings	[ ]
Total duration	[ ] min
Sampling frequency	[ ] Hz
Bit depth	[16] bit
Channels	[1 (mono)]
Minimum duration	[ ] s
Maximum duration	[ ] s
Average duration	[ ] s
Training set	[ ] recordings
Validation set	[ ] recordings

[Insert duration_histogram.png here.]

Figure 3. Histogram of recording durations for speaker [F3], emotion [Z].

Most recordings last between [ ] and [ ] s. [Describe the shape: e.g. there are many short phrases and only a few long sentences.] Very short or very long recordings are less useful for training, so they could be removed before fine-tuning. [Explain why you picked this speaker and emotion, e.g. it had the most recordings in the downloaded part, and neutral speech is the easiest starting point for training.]

6. Conclusions

1. The pretrained YourTTS model supports English, French and Brazilian Portuguese. Lithuanian is not supported.
2. For English, zero-shot synthesis worked [well]: the speech was understandable and the voice was [similar] to mine. The main differences from natural speech were [duration / speaking rate], a limited bandwidth (up to 8 kHz, because the output is 16 kHz) and a [flatter] F0 contour.

3. For Lithuanian with the English setting, the voice was still copied, but the pronunciation was [poor]. The model used English pronunciation rules, and Lithuanian letters such as "š" and "ė" were [dropped / mispronounced]. Zero-shot synthesis is therefore not enough for an unsupported language.
4. To make the model speak Lithuanian properly, it needs to be fine-tuned on Lithuanian data. For this, a dataset of [ ] recordings ([ ] min) from speaker [F3], emotion [Z], was prepared from the EMO corpus and split into [ ] training and [ ] validation recordings in the Coqui TTS format.